

On the Energy Efficiency of Device Discovery in Mobile Opportunistic Networks: A Systematic Approach

Bo Han, Jian Li, and Aravind Srinivasan, *Fellow, IEEE*

Abstract—In this paper, we propose an energy efficient device discovery protocol, *eDiscovery*, as the first step to bootstrapping opportunistic communications for *smartphones*, the most popular mobile devices. We chose Bluetooth over WiFi as the underlying wireless technology of device discovery, based on our measurement study of their operational power at different states on smartphones. *eDiscovery* adaptively changes the duration and interval of Bluetooth inquiry in dynamic environments, by leveraging history information of discovered peers. We implement a prototype of *eDiscovery* on Nokia N900 smartphones and evaluate its performance in three different environments. To the best of our knowledge, we are the first to conduct extensive performance evaluation of Bluetooth device discovery *in the wild*. Our experimental results demonstrate that compared with a scheme with constant inquiry duration and interval, *eDiscovery* can save around 44 percent energy at the expense of discovering only about 21 percent less peers. The results also show that *eDiscovery* performs better than other existing schemes, by discovering more peers and consuming less energy. We also verify the experimental results through extensive simulation studies in the ns-2 simulator.

Index Terms—Device discovery, opportunistic communications, energy efficiency, smartphones, Bluetooth

1 INTRODUCTION

MOBILITY itself is a significant problem in mobile networking. On the one hand, protocols designed for mobile networks should solve the challenges caused by the mobility of wireless devices. For example, routing protocols, such as dynamic source routing (DSR) [16], are required to handle frequent routing changes and reduce the corresponding communication overhead. On the other hand, mobility can increase the capacity of wireless networks through opportunistic communications [13], where mobile devices moving into wireless range of each other can exchange information *opportunistically* during their periods of contact [7], [21].

Opportunistic communications have been widely explored in delay-tolerant networks [32], mobile social applications [21], [31] and mobile advertising [1], to facilitate message forwarding, media sharing and location-based services. Meanwhile, there are more and more applications leveraging opportunistic communications for various purposes. For example, LoKast¹ is an iPhone application that

provides mobile social networking services by discovering and sharing media content among users in proximity. Nintendo 3DS's StreetPass² enables players to exchange game data with other users they pass on the street, through the direct device-to-device communication between 3DS systems. Other similar applications include Sony PS Vita's Near and Apple's iGroups.

Device discovery is essentially the *first* step of opportunistic communications. However, there are very few practical protocols proposed for it and most of the existing work mainly utilizes (trace-driven) simulation to evaluate the performance of various device discovery protocols [9], [29]. Moreover, although there are several real-world mobility traces in the CRAWDAD repository³ which were collected using Bluetooth device discovery, most of them used very simple discovery protocols with *constant* inquiry duration and interval. A recently proposed opportunistic Twitter application [24] also uses a 2-minute inquiry interval for Bluetooth device discovery. It is known that these kinds of discovery protocols are not energy efficient [29] and thus may not be desirable for power-constrained mobile devices, such as smartphones. In this paper, we bridge this gap by developing an *energy-aware* device discovery protocol for smartphone-based opportunistic communications and evaluating its performance in practice.

There are two major challenges in designing, implementing and evaluating energy efficient device discovery protocols for smartphones. First, the selection of underlying communication technology is complicated by the multiple wireless interfaces on smartphones, such as

1. <http://www.lokast.com/>.

• B. Han is with AT&T Labs – Research, 1 AT&T Way, Bedminster, NJ 07921. E-mail: bohan@research.att.com.
• J. Li is with the Institute for Interdisciplinary Information Sciences, Tsinghua University, Beijing 100084, China. E-mail: lijian83@mail.tsinghua.edu.cn.
• A. Srinivasan is with the Department of Computer Science and the Institute for Advanced Computer Studies, University of Maryland, College Park, MD 20742. E-mail: srin@cs.umd.edu.

Manuscript received 15 Jan. 2014; revised 9 June 2014; accepted 21 June 2014.
Date of publication 7 July 2014; date of current version 2 Mar. 2015.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TMC.2014.2333749

Authorized licensed use limited to: Columbia University Libraries. Downloaded on July 06, 2026 at 02:36:33 UTC from IEEE Xplore. Restrictions apply.

1536-1233 © 2014 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

2. <http://www.nintendo.com/3ds/features/>.

3. <http://crawdad.cs.dartmouth.edu/>.

Bluetooth and WiFi (a.k.a., IEEE 802.11).⁴ Although Bluetooth is a low-power radio, its device discovery duration is much longer than WiFi (~ 10 s for Bluetooth versus ~ 1 s for WiFi active scanning), which may cause more energy consumption on smartphones. Similarly, WiFi is known to be power-hungry for mobile devices [22], [26]. Thus, it is not clear which of them is more suitable for device discovery on smartphones.

Second, given the dynamic nature of human mobility, we need to adaptively tune device discovery parameters, such as inquiry duration and interval, to reduce smartphone energy consumption. Schemes with constant inquiry intervals have been proven to be optimal for minimizing discovery-missing probability [29]. However, their energy consumption is usually higher than the adaptive ones, which may miss more devices during discovery procedures. Therefore, there is a tradeoff between energy consumption and discovery-missing probability.

We make the following contributions in this paper.

- We present a systematic measurement study of the energy consumption of Bluetooth and WiFi *device discovery* on smartphones, by measuring both the electrical power at various states and the discovery duration (Section 4). Our results show that the energy consumption depends on the number of discovered peers. Based on our measurement results, we chose Bluetooth as the underlying wireless technology, because even its high-power state consumes less energy than the low-power state of WiFi during device discovery. We emphasize that although previous works have studied the power of Bluetooth/WiFi devices [9], [11], [22], they either focus on only Bluetooth [9] or ignore the duration of device discovery [11], [22], without which it is hard to evaluate the *energy consumption* of these devices.
- We design an energy-aware device discovery protocol, named *eDiscovery*, as the first and very important step to bootstrapping smartphone-based opportunistic communications (Section 6). By trading energy consumption for a limited discovery loss, we demonstrate that *eDiscovery* is highly effective in saving energy on smartphones. *eDiscovery* dynamically tunes the discovery duration and interval according to history information of the number of discovered peers. It also introduces randomization into device discovery, in order to explore the search space further.
- Our major contribution is an extensive performance evaluation of *eDiscovery* and other existing device discovery protocols in different realistic environments, through a prototype implementation on Nokia N900 smartphones (Section 7). We conduct experiments in a university campus, a metro station and a shopping center. Our experimental results verify the effectiveness of *eDiscovery* in practice. Compared with the STAR protocol proposed by Wang et al. [29], *eDiscovery* consumes less energy

and discovers more peers. *eDiscovery* also performs better than another protocol in the literature.

Compared with its preliminary version [15], this paper makes two new contributions. First, we add the theoretical analysis of device-discovery missing probability (Section 5), which motivates us to design the *eDiscovery* protocol. Second, we port the implementation of *eDiscovery* into the ns-2 simulator enhanced with the UCBD Bluetooth module⁵ and offer a detailed evaluation of its parameters (Section 8). We also compare the performance of *eDiscovery* and STAR with more network topologies using ns-2 simulations.

2 DEVICE DISCOVERY IN BLUETOOTH AND WIFI

In the following, we discuss device discovery of Bluetooth and WiFi, the two most commonly available local wireless communication technologies on smartphones.

2.1 Bluetooth

The Bluetooth specification (Version 2.1) [3] defines all layers of a typical network protocol stack, from the base-band radio layer to the application layer. Bluetooth operates in the 2.4 GHz ISM (Industrial, Scientific and Medical) frequency band, shared with other devices such as IEEE 802.11 stations, baby monitors and microwave ovens [12]. Therefore, it uses Frequency-Hopping Spread Spectrum (FHSS) to avoid cross-technology interference, by randomly changing its operating frequency bands. Bluetooth has 79 frequency bands (1 MHz width) in the range 2402-2480 MHz and the duration of a Bluetooth time slot is 625 μ s. In the following we focus on device discovery and refer interested readers to Smith et al. [27] for further study of the Bluetooth protocol stack.

During device discovery, an inquiring device sends out inquiry messages periodically and waits for responses, and a scanning device listens to wireless channels and sends back responses after receiving inquiries [3]. The inquiring device uses two trains of 16 frequency bands each, selected from 79 bands. The 32 bands of these two trains are selected according to a pseudo-random scheme and a Bluetooth device switches its trains every 2.56 seconds. In every time slot, the inquiring device sends out two inquiry messages on two different frequency bands and waits for response messages on the same frequency bands during the next time slot. After a device receives an inquiry message, it will wait for 625 μ s (i.e., the duration of a time slot) before sending out a response message on the same frequency band, which completes the device discovery procedure. For scanning devices, Bluetooth controls their scanning duration and frequency with two parameters, scan window and scan interval.

2.2 Bluetooth Low Energy (LE)

Bluetooth Low Energy [4] operates in the 2400-2483.5 MHz frequency band and divides this band into 40 channels with 2 MHz width, instead of 79 channels with 1 MHz width in the classic Bluetooth. Three out of these 40 channels, with

4. We prefer Bluetooth and WiFi to 3G, as they are local communication technologies with almost no monetary cost.

5. <http://www.cs.uc.edu/~cdmc/ucbt/>.

channel indexes 37, 38, and 39, are used for advertising, and the rest are data channels.

Differently from the classic Bluetooth, the LE system leverages these advertising channels for device discovery and connection establishment. Among the five states defined in Bluetooth LE, three of them are related to device discovery: advertising, scanning and initiating states (the rest two are standby and connection states). After a device enters the advertising state (directed by the host machine), it sends out one or more advertising packets that contains its device address on the advertising channels. These advertising packets compose the so-called advertising events. The time between the start of two consecutive advertising events is defined as the sum of a constant *advInterval*, which should be an integer multiple of 625 μ s and in the range of 20 ms to 10.24 s, and a pseudo-random value *advDelay* in the range of 0 to 10 ms. A device in either scanning or initiating state listens on an advertising channel with the duration of *scanWindow* and the interval *scanInterval* (i.e., the interval between the start of two consecutive scan windows). The *scanWindow* and *scanInterval* parameters should not be greater than 10.24 s.

2.3 WiFi

The key concept of device discovery in WiFi is well understood. WiFi stations in infrastructure and ad-hoc modes periodically (100 ms by default) send out Beacon messages to announce the presence of a network. A Beacon message includes information such as service set identifier (SSID) and capability information. The WiFi interfaces of mobile phones should operate in ad-hoc mode and form an independent basic service set (IBSS) to support opportunistic communications, since infrastructure-mode interfaces cannot form a network and thus cannot communicate directly. Besides sending out Beacon messages, a WiFi interface also scans wireless channels to discover peers.

There are two types of WiFi scanning, passive and active. In passive scanning, a WiFi interface listens for Beacon messages on each channel, broadcasted by its peers at regular intervals. It periodically switches channels, but does not send any probe request message. During active scanning, a WiFi interface actively searches for its peers, by broadcasting probe request messages on each possible operating channel (channels 1 to 11 in North America). It then waits for probe response messages from its peers, which include information similar to that in Beacon messages.

We prefer active scanning to passive scanning for device discovery of opportunistic networks for two reasons. First, although passive scanning has the advantage of not broadcasting probe request messages, it dwells on each channel longer than active scanning, to collect Beacon messages from peers, and thus may consume more energy. Second, an ad-hoc mode interface may skip the sending of Beacon messages and thus make itself not discoverable by passive scanning, when it tries to scan for other peers with the same SSID (which happens frequently when it is the only station in an IBSS).

3 RELATED WORK

In this section, we briefly review the literature of device discovery in wireless networks.

3.1 Wireless Device Discovery in General

Device discovery has been studied in various wireless networks, such as ad-hoc networks [20], [28], sensor networks [10], [17] and delay-tolerant networks [29].

Neighbor/device discovery is one of the first steps to initialize large wireless networks. McGlynn and Borbash [20] examine the problem of neighbor discovery during the deployment of static ad-hoc networks, where the discovery may last only a few minutes. Vasudevan et al. [28] show that an existing ALOHA-like neighbor discovery algorithm reduces to the classical Coupon Collector's Problem when nodes are not capable of collision detection. They also propose an improved algorithm based on receiver status feedback when nodes have a collision detection mechanism. Cohen and Kapchits [6] investigate a slightly different neighbor discovery problem in asynchronous sensor networks. Instead of study the initial neighbor discovery, they are interested in continuous neighbor discovery after the initial discovery phase. Unlike the above works that are based on abstract communication models, our focus is practical Bluetooth device discovery for smartphone-based opportunistic communications.

Dutta and Culler [10] propose an asynchronous neighbor discovery protocol, called Disco, for mobile sensing applications. U-Connect [17] is another asynchronous neighbor discovery protocol for mobile sensor networks that selects carefully the time slots to perform discovery and that has been proven theoretically better than Disco. Recently, Bakht et al. [2] propose Searchlight, a protocol that combines both deterministic and probabilistic approaches to further reduce the discovery latency for mobile social applications. Disco, U-Connect and Searchlight mainly aim to achieve a tradeoff between discovery latency and energy consumption. For example, U-Connect uses the power-latency product metric for performance evaluation. Differently from them, we are interested in the tradeoff between energy consumption and discovery-missing probability.

The goal of eDiscovery is similar in spirit to that of Wang et al. [29] who investigate the tradeoff between the contact probing frequency (which determines energy consumption) and the missing probability of a contact for delay tolerant applications. They design a contact probing algorithm, named short term arrival rate (STAR), to dynamically change the contact probing frequency. Specifically, STAR estimates the peer arriving rate over a short time, which decays slowly, and calculates the probing frequency based on the estimation. Without specifying the communication technologies, they assume that every probing message is just an impulse and consumes no time. We compare the performance of eDiscovery with STAR through extensive real-world experiments and simulation studies.

FlashLinQ [30] is a synchronous wireless network architecture developed by Qualcomm for direct device-to-device communication over licensed spectrum. Although FlashLinQ may be more energy efficient than Bluetooth and WiFi, given its clean-slate design for ad hoc networks, it requires special purpose hardware and also operates in licensed spectrum. Unlike from FlashLinQ, we aim to design and implement device discovery protocols using existing hardware and communication technologies available on commercial smartphones.

3.2 Bluetooth Device Discovery

Bluetooth specifies a detailed device discovery protocol [3]. Salonidis et al. [25] identify the bottlenecks of asymmetric device-discovery delay of Bluetooth and introduce a randomized symmetric discovery protocol to reduce this delay. Based on Bluetooth specification v1.1, Peterson et al. [23] derive rigorous expressions for the inquiry-time probability distribution of two Bluetooth devices that want to discover each other. Chakraborty et al. [5] present an analytical model of the time of Bluetooth device discovery protocol. Liberatore et al. [18] solve the problem of long discovery duration of Bluetooth due to its half-duplex discovery process by the addition of another Bluetooth radio.

Drula et al. [9] study how to select Bluetooth device discovery parameters according to the mobility context and thus reduce the energy consumption of device discovery. They present two algorithms that adjust these parameters based on recent activity level (referred as RAL) and the location of previous contacts, and evaluate their performance through simulations. RAL defines several inquiry modes based on parameters, such as inquiry duration and interval, and switches to a more aggressive mode whenever another device is discovered. In our previous work [14], we compare energy consumption of Bluetooth and WiFi device discovery on Nokia N900 smartphones, using battery life as a metric. We evaluate the Bluetooth device-discovery probability in an office environment using a static phone and a moving phone.

Besides the above works, although there is a large body of literature about Bluetooth device discovery, most of them focus on the improvements of discovery latency between two Bluetooth devices by tuning various parameters or changing the protocol itself, which may not be feasible to implement on smartphones. The focus of this paper is about the performance evaluation of an energy-aware device discovery protocol *in the wild* through a prototype implementation on smartphones.

Bluetooth device discovery has been an important component in opportunistic networks. We refer interested readers to a preliminary version of this paper [15] for a literature review in that area.

4 ENERGY CONSUMPTION OF DEVICE DISCOVERY

In this section, we measure the power and energy consumption of Bluetooth and WiFi device discovery on smartphones. Based on the experimental results, we chose Bluetooth as the communication technology. Although previous work has measured energy consumption of WiFi and Bluetooth devices several years ago [9], [22], these results may be invalid given the rapid development of battery and wireless technologies [11]. Friedman et al. [11] have recently studied the power of Bluetooth scanning and WiFi search. However, they overlook the duration of device discovery which determines the energy consumption on smartphones. Furthermore, their measurements are for station mode WiFi interfaces and demonstrate inconsistent results about WiFi device discovery. To the best of our knowledge, there is no systematic study of smartphone energy consumption of Bluetooth and WiFi device discovery.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on July 06, 2026 at 02:36:33 UTC from IEEE Xplore. Restrictions apply.

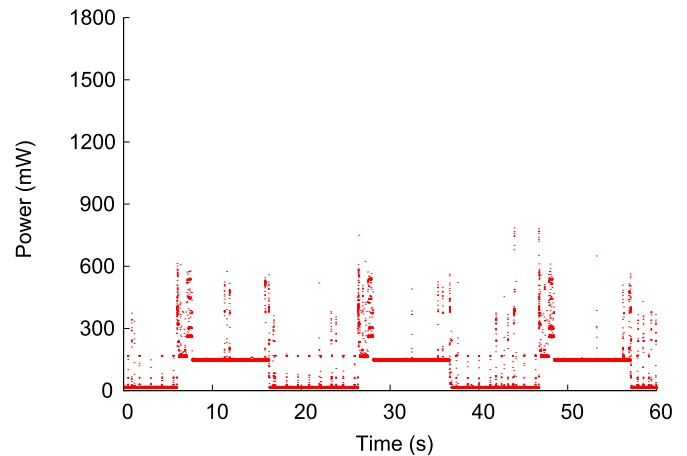


Fig. 1. A 60-second snapshot of the temporal power of periodic Bluetooth device discovery with 10-second interval. The smartphone under test is a Nokia N900 smartphone.

4.1 Measurement Setup

We measure the electrical power of two states of Bluetooth and WiFi device discovery, idle (i.e., inquiry interval) and active probing, on Nokia N900 smartphones using the Monsoon power monitor.⁶ The default OS of Nokia N900, Maemo 5, is an open source Linux distribution (kernel version 2.6.28). Its WiFi chipset is Texas Instruments WL1251 using the wl12xx device driver.⁷ Its Bluetooth chipset is Broadcom BCM2048. We use BlueZ,⁸ the default Bluetooth protocol stack of most Linux distributions, to run Bluetooth device discovery experiments. During the measurements, we redirect standard output to `\dev \null` and turn the screen off to minimize their impact on the measurement results. We report the average result and the 95 percent confidence intervals for each configuration over 10 runs in this section. Note that all results of power measurements in this paper include the baseline power of the smartphone under test.

4.2 Bluetooth

We present a 60-second snapshot of the power of Bluetooth device discovery in Fig. 1. We perform the experiments by running `hcitool`, a tool that can send commands, such as `inq` (inquiry), to Bluetooth devices. We use the `flush` option to clear the cache of previously discovered devices before each inquiry. During the measurements, the phone queries neighboring Bluetooth devices periodically with a 10-second interval. When there is no neighboring device, the average power of Bluetooth inquiry over 10 runs is ~ 162.03 mW (standard deviation: 2.12 mW). During the idle states, the Bluetooth radio is in discoverable mode with average power ~ 16.54 mW (standard deviation: 1.11 mW).

The average power of Bluetooth device discovery is affected by the number of neighboring devices. We repeat the experiments with the number of neighboring Bluetooth devices increasing from 0 to 6 and summarize the results in Table 1. As we can see from this table, when there is one neighboring device, the average power increases to around

6. <http://www.monsoon.com/LabEquipment/PowerMonitor/>.

7. <http://linuxwireless.org/en/users/Drivers/wl12xx>.

8. <http://www.bluez.org/>.

TABLE 1
The Electrical Power (in mW) of Bluetooth Device Discovery with Different Numbers of Peers

# of Devices	Average	95% Confidence Interval
0	162.03	(160.72, 163.34)
1	227.06	(219.42, 234.70)
2	247.72	(242.39, 253.05)
3	248.91	(243.02, 254.80)
4	248.59	(246.63, 250.55)
5	256.02	(252.96, 259.08)
6	253.05	(249.63, 256.47)

227.06 mW, due to the reception of response messages of Bluetooth inquiry. When there are more than one neighboring devices, the average power increases to about 250 mW. Defined in the standard [3], the duration of Bluetooth device discovery should be a multiple of 1.28 seconds and the recommended default value is 10.24 seconds, which we used in the measurements. Fig. 1 shows clearly the configured Bluetooth device discovery duration and interval.

4.3 WiFi

We present another 60-second snapshot of the power of WiFi device discovery in Fig. 2. We perform the experiments by running *iwlist*, a tool that shows the list of access points and ad-hoc cells in range through active scanning. During the measurements, the phone scans neighboring devices periodically also with a 10-second interval, which can be clearly identified in Fig. 2. The average power of WiFi active scanning over 10 runs is ~ 836.65 mW (standard deviation: 8.98 mW). Even during scanning intervals, the average power is ~ 791.02 mW (standard deviation: 5.23 mW), because the WiFi radio is in ad-hoc mode and sends out Beacon messages with 100 ms intervals.

Differently from Bluetooth, the duration of WiFi active scanning is not constant and may depend on the number of operation channels and the amount of neighboring peers. We measure the duration of WiFi device discovery in three different environments: a campus office building, an apartment, and a national park, and summarize the results in

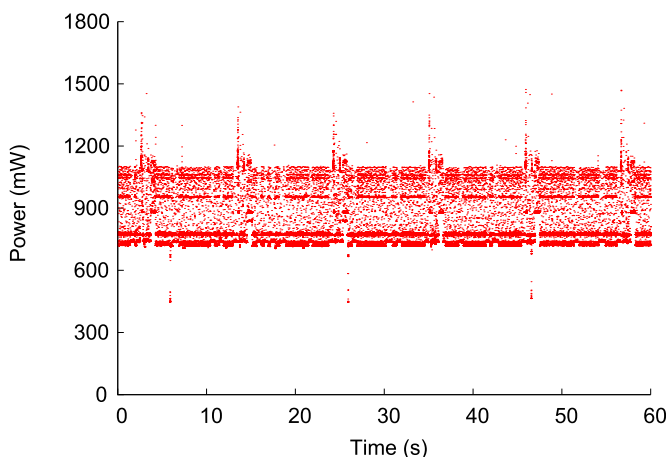


Fig. 2. A 60-second snapshot of the temporal power of periodic WiFi device discovery with 10-second interval. The smartphone under test is a Nokia N900 smartphone.

TABLE 2
The Average Number of Discovered Peers and Duration of WiFi Device Discovery in Three Environments

Environment	# of peers	duration (s)
Office	43.52 (42.48, 44.56)	1.07 (1.04, 1.10)
Home	14.02 (13.75, 14.29)	0.87 (0.86, 0.88)
Park	0.01 (n/a)	0.52 (0.51, 0.53)

The numbers in the parentheses are the 95 percent confidence intervals.

Table 2. In each environment, we repeat the experiments 100 times and report the average values and the 95 percent confidence intervals. As we can see from this table, when the number of discovered peers increases, the duration of WiFi device discovery grows from ~ 0.52 seconds to ~ 1.07 seconds, which is much shorter than the duration of Bluetooth inquiry.

4.4 Energy Consumption

We summarize the average power of Bluetooth (with six neighboring devices) and WiFi device discovery in Table 3. Suppose the power is P_{idle} for the idle state and P_{probe} for the inquiry/scan state of Bluetooth/WiFi devices, the duration of Bluetooth inquiry/WiFi scan is T_{probe} and the inquiry/scan interval is T_{idle} . Then the estimated energy consumption is

$$E = T_{idle} \cdot P_{idle} + T_{probe} \cdot P_{probe}.$$

Given the high power of WiFi device discovery in both active probing and idle states, we prefer Bluetooth to WiFi for device discovery of smartphone-based opportunistic communications. We note that no matter how long the duration of Bluetooth inquiry is, the overall energy consumption of Bluetooth device discovery should always be lower than that of WiFi, because the power of Bluetooth inquiry is even lower than that of the WiFi idle state (253.05 versus 791.02 mW). To perform device discovery, the major problem of WiFi ad-hoc mode is that the radio needs to send out Beacon messages periodically and power saving mechanisms for WiFi ad-hoc mode are not available on most mobile phones [26].

Although the communication range of WiFi is longer than Bluetooth and may discover more peers, making its device discovery energy efficient requires substantial modifications of the WiFi protocol, which may not be feasible on most smartphones. In this paper, we aim to design a device discovery protocol without changing the underlying communication protocol and thus make its deployment easy. This is another reason why we chose Bluetooth over WiFi. However, we emphasize that if energy consumption is not a major concern and the design goal is to discover as many

TABLE 3
The Average Power of Bluetooth and WiFi Device Discovery in mW

	P_{idle}	P_{probe}
Bluetooth	16.54 (15.85, 17.23)	253.05 (249.63, 256.47)
WiFi	791.02 (787.78, 794.26)	836.65 (831.08, 842.22)

The numbers in the parentheses are the 95 percent confidence intervals.

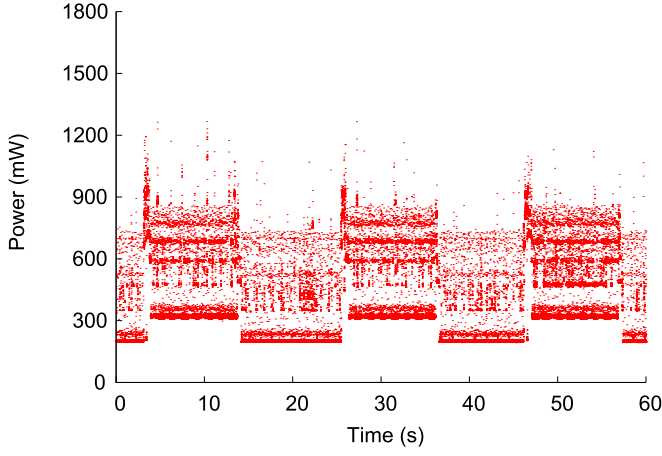


Fig. 3. A 60-second snapshot of the temporal power of periodic Bluetooth device discovery with 10-second interval. The smartphone under test is a HTC Hero smartphone (Android 1.5).

peers as possible or to transfer a large amount of data efficiently, we should use WiFi as the underlying communication protocol (which is out of the scope of this paper), because it has a larger coverage area.

4.5 Android Smartphones

We also measured the power of Bluetooth and WiFi device discovery using a HTC Hero smartphone with Android 1.5. We plot the results in Fig. 3 for Bluetooth and Fig. 4 for WiFi. On this smartphone, the average power is 432.84 mW (standard deviation: 7.86 mW) for Bluetooth inquiry and 900.25 mW (standard deviation: 21.54 mW) for WiFi scan. There are two differences of the experiments on the Nokia N900 and HTC Hero smartphones. First, the experiments on HTC Hero were performed with the screen on due to the operational requirements and thus the baseline power of HTC Hero is higher than that of Nokia N900. Second, the WiFi interface on HTC Hero does not support ad-hoc mode and we cannot measure the average power P_{idle} on it. However, these results still clearly show the significant power difference (467.41 mW) of Bluetooth inquiry and WiFi scan.

5 DEVICE DISCOVERY MISSING PROBABILITY

In this section, we analyze the missing probability of a device discovery protocol with constant Bluetooth inquiry window and interval (referred as Constant in the following).

First, we introduce some notations. For a given device i , we assume that the contact durations $t_D(i)$ are independent and identically distributed random variables with common probability density function (PDF) $p(x) = \frac{d}{dx} \Pr[t_D \leq x]$. We assume the inter-contact time (the time between subsequent contacts) $t_C(i)$ are stationary random variables.

If a scanning device is in the discovery/contact range of an inquiring device for L seconds, it can be discovered with probability $R(L)$. We can easily derive $R(L)$ from the analysis of the probability distribution of the inquiry time for Bluetooth devices by Peterson et al. [23]. For Bluetooth device discovery, $R(L)$ is a monotonically increasing function of L . Let $\bar{R}(L) = 1 - R(L)$. Assume that for different devices, or for the same device in different contact intervals,

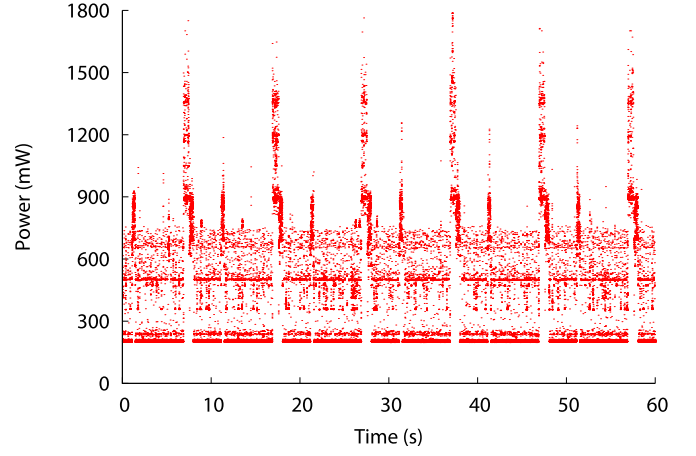


Fig. 4. A 60-second snapshot of the temporal power of periodic WiFi device discovery with 10-second interval. The smartphone under test is a HTC Hero smartphone (Android 1.5).

the discovery probabilities are independent of each other. For any real number L , we let $L_+ = \max\{L, 0\}$. We use P_{miss} to denote the probability that a contact is missed (i.e., the scanning device is not discovered by the inquiring device).

Now, we analyze P_{miss} for the Constant protocol that repeatedly performs Bluetooth inquiry for W seconds and sleeps for next V seconds. Let $T = W + V$. Suppose the scanning device arrives at time $nT + x$ for some very large integer n and $x \in [0, T]$ and the contact duration t_D is a constant real L . We use $P_{miss}(x, L)$ to denote the missing probability under the above condition. Let $k = \frac{L+x-T}{T}$. Based on the value of k , we distinguish the following two cases, as shown in Fig. 5.

- 1) If $k < 0$ (i.e., $x + L < T$, contact #1 in Fig. 5) which means the contact ends within the period T , it is easy to see that

$$P_{miss}(x, L) = 1 - R(\min\{L, (W - x)_+\}) \\ = \bar{R}(\min\{L, (W - x)_+\}).$$

- 2) If $k > 0$ (e.g., contact #2 in Fig. 5), there may be more than one inquiry cycles (inquiry window plus inquiry interval). Let us consider them one by one. In $(nT, (n+1)T)$, the scanning device is not discovered by the inquiring device with probability $\bar{R}((W - x)_+)$. In each of the next $[k]$ cycles, the two devices are in the contact range for W seconds and

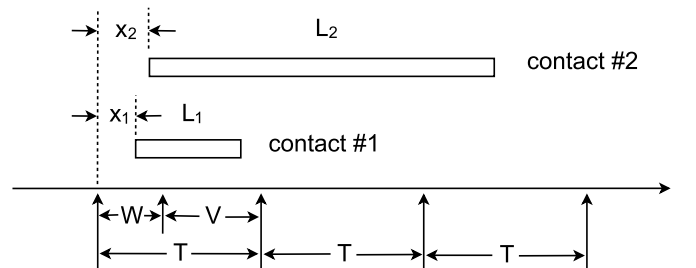


Fig. 5. Two contact cases for analyzing the device discovery missing probability.

the missing probability is $\bar{R}(W)$. In the last possible cycle, the contact interval is of length $\min(y, W)$, where $y = L + x - T - \lfloor k \rfloor T$. Therefore,

$$P_{miss}(x, L) = \bar{R}((W - x)_+) \bar{R}(W)^{\lfloor k \rfloor} \bar{R}(\min(y, W)).$$

If the inter-contact time follows a nonlattice distribution, by Blackwell's Theorem in renewal theory [29], we have that, when $n \rightarrow \infty$,

$$\begin{aligned} P_{miss} &= \int_0^T \int_0^\infty P_{miss}(x, L) p(L) dL dx \\ &= \int_0^T \int_0^{T-x} \bar{R}(\min\{L, (W - x)_+\}) p(L) dL dx \\ &+ \int_0^T \int_{T-x}^\infty \bar{R}((W - x)_+) \bar{R}(W)^{\lfloor k \rfloor} \bar{R}(\min(y, W)) p(L) dL dx. \end{aligned}$$

Similar with the analysis by Wang et al. [29], the missing probability is independent of the inter-contact time distribution. The major difference is that we also consider the inquiry window duration in our analysis, besides the contact duration and inquiry interval. A key observation here is that the missing probability is also independent of device density and is solely determined by the values of W and V . Therefore, if we dynamically tune W and V to decrease (increase) the missing probability when the device density is high (low), we may be able to increase the overall device-discovery probability. For example, if we fix the missing probability at 0.2, for two adjacent areas with density 100 and 10 we can discover 88 devices. However, if we decrease the missing probability to 0.1 for the dense area and increase it to 0.3 for the sparse area, we can discover 97 devices. This observation motivates us to design the eDiscovery protocol in the next section.

6 eDISCOVERY DESIGN

In this section, we present eDiscovery, an energy-aware device discovery protocol that adaptively changes the duration and probing interval of Bluetooth inquiry.

The major design principle of eDiscovery is to reduce smartphone energy consumption of device discovery, while not missing too many peers. To achieve this goal, we dynamically change the duration and interval of Bluetooth device discovery, based on the number of discovered peers. In theory, if a mobile device knows the density of its peers at any given time, it may be able to select the optimal values for these two Bluetooth device discovery parameters. However, in practice it is hard to estimate this density, especially in dynamic environments, such as shopping malls and train stations. Therefore, we present a heuristic adaptive inquiry approach of eDiscovery in Algorithm 1.

There are two approaches to control the duration of Bluetooth device discovery: (1) specifying the length of the inquiry window explicitly or (2) specifying the number of received responses before device discovery stops. Accordingly, there are two parameters of `hci_inquiry`, the device discovery function of BlueZ, `inquiry_window` and `num_responses`. This function stops inquiry after

$1.28 \times \text{inquiry_window}$ seconds or it has received `num_responses` inquiry responses.

Algorithm 1. Adaptive Inquiry Algorithm of eDiscovery

```

1: inquiry_window = base_W, inquiry_interval = base_I;
2: while (TRUE) do
3:   peers = hci_inquiry(inquiry_window, MAX_RSP);
4:   if (peers >  $N$ ) then
5:     inquiry_window = base_W;
6:   else
7:     inquiry_window = small_W +  $r$ ;
8:   end if
9:   if (peers == 0 and last_peers == 0) then
10:    inquiry_interval += inc_NP +  $r$ ;
11:   else if (peers < 0 and last_peers == 0) then
12:    inquiry_interval = base_I +  $r$ ;
13:   else if (peers > last_peers and inquiry_interval -  $I$  is
in the valid range) then
14:    inquiry_interval - =  $I$ ;
15:   else if (peers < last_peers and inquiry_interval +  $I$  is
in the valid range) then
16:    inquiry_interval +=  $I$ ;
17:   end if
18:   last_peers = peers;
19:   sleep(inquiry_interval);
20: end while

```

We focus on the control of the inquiry window in this paper, as it is hard to predict the number of neighboring peers in practice. Moreover, a peer can respond to an inquiry more than once. Suppose there are three neighboring peers, A, B, and C, and we set `num_responses` to be 3. If all the first three responses are sent by peer A, device discovery will stop after receiving them and thus discover only peer A. We note that eDiscovery sits between mobile applications and Bluetooth device discovery and thus the contention/collision of Bluetooth device discovery messages are resolved at the MAC layer of Bluetooth protocol stack.

The two key parameters in Algorithm 1 of eDiscovery are the threshold of the number of discovered peers N and the increment/decrement of inquiry interval I . The outputs of Algorithm 1 are `inquiry_window` and `inquiry_interval`, which control the duration and interval of Bluetooth inquiry. The main body of this algorithm is a while loop that performs Bluetooth inquiry $1.28 * \text{inquiry_window}$ seconds and then sleeps `inquiry_interval` seconds.

After each Bluetooth inquiry, we adapt the values of `inquiry_window` based on the number of discovered peers. If this number is larger than N , we keep the default initial value `base_W`, aiming to discover more peers. If it is smaller or equal to N , we set the next `inquiry_window` to be `small_W` + r , where r is defined as

$$r = \begin{cases} 1 & \text{with probability } (1 - p)/2 \\ 0 & \text{with probability } p \\ -1 & \text{with probability } (1 - p)/2. \end{cases}$$

By changing `inquiry_window` in this way, we can reduce the duration of Bluetooth inquiry and thus save energy

on smartphones when the number of neighboring peers is small.

We adapt the value of *inquiry_interval* to the number of discovered peers in a similar way. When a smartphone discovers no peers for two consecutive inquiries, we increase *inquiry_interval* by $inc_NP + r$ and reset it to $base_I + r$ after the smartphone discovers new peers. Moreover, if the current number of discovered peers is larger than the previous one, we decrease *inquiry_interval* by I , and vice versa. An implication of this algorithm is that *inquiry_interval* will not change if the number of discovered peers does not vary. We allow *inquiry_interval* to vary between 10-200 seconds. The random variable r is refreshed for every inquiry. We use it for improving the robustness of eDiscovery for dynamic environments. Furthermore, it can avoid synchronization of Bluetooth inquiry which may make Bluetooth devices not be able to discover each other [14], [23].

The intuition behind these adaptations is that we can reduce inquiry duration and increase inquiry interval when the number of neighboring peers is small, because doing this will not miss too many peers. By changing the values of N and I , we can achieve different tradeoff between the number of discovered peers and smartphone energy consumption. Smaller N and I lead to more aggressive Bluetooth inquiry, which may discover more peers but also consume more energy on smartphones.

We list the default values of the parameters of Algorithm 1 in Table 4. These values are not set arbitrarily. We set the initial *inquiry_window* to be 8 (i.e., $8 * 1.28 = 10.24$ seconds) because it is the default standard value of Bluetooth inquiry. We set *MAX_RSP* to be 255 (the suggested value in BlueZ protocol stack). We set *small_W* to be 5. Thus when the number of discovered peers is smaller than N , the smallest inquiry window $5 + r$ would be 4, as this is the minimum inquiry window to perform a complete scan of all possible frequency bands. Moreover, Peterson et al. [23] demonstrate that by setting the *inquiry_window* to be 4, a Bluetooth device can locate 99 percent of neighboring devices within its transmission range in a static environment. When deciding the probability p in r , essentially, we want to set the parameters to be their default values under certain conditions with a high probability (by default $p = 0.8$) and slightly change their values by 1 with a low probability.

We evaluate the performance of eDiscovery with different values of N and I in Section 7. We also evaluate how other parameters, such as *base_W*, *base_I* and the choice of random variable r , affect the performance of eDiscovery in Section 8.1. Moreover, we evaluate various solutions of changing inquiry interval based on the difference of discovered peers, by replacing line 14 in Algorithm 1 with

inquiry_interval− = $ceil(\alpha \times abs(peers - last_peers))$;

and line 16 with

inquiry_interval+ = $ceil(\alpha \times abs(peers - last_peers))$;

where α is a parameter to further tune the value of *inquiry_interval*. Finally, we note that there may be a diminishing return for some applications for which the change from discovering nothing to one peer is more important than that from discovering, for example, 10 to

TABLE 4
The Parameters in Algorithm 1 and Their Default Values

Parameter	Default	Description
N	5	Threshold of discovered peers
I	1	Increment of inquiry interval
<i>base_W</i>	8	Base of inquiry window
<i>base_I</i>	10	Base of inquiry interval
<i>MAX_RSP</i>	255	Maximum number of scanned peers
<i>small_W</i>	5	Smaller inquiry window
<i>inc_NP</i>	10	Increment of interval when no peers
r	0	Random variable for robustness
p	0.8	Probability of $r = 0$

11 peers. In this case, we need to invest more energy when the number of peers is small, the opposite behavior of our eDiscovery protocol.

7 EXPERIMENTAL EVALUATION

In this section, we evaluate the performance of our proposed eDiscovery protocol through a prototype implementation on Nokia N900 smartphones and compare it with other schemes. Although previous work has evaluated device discovery protocols using simulations [9], [29], a recent study demonstrates that even contact-based simulations using real-world mobility traces may not be able to accurately evaluate the performance of opportunistic networks [24]. Moreover, it is also not clear how Bluetooth device discovery performs in the wild, under cross-technology interference [12].

We implement eDiscovery in C language using the BlueZ protocol stack and compare its performance with three other approaches: the Constant protocol in Section 5, the STAR algorithm by Wang et al. [29] and the RAL scheme by Drula et al. [9]. The two metrics we are interested in are the ratio of discovered peers, compared to the ground truth, and the estimated energy consumption. To get the ground truth, we perform Bluetooth inquiry with the default 10.24-second duration continuously. Based on the ground truth, we can know how many peers Constant can discover by aggregating the inquiries in the ground truth with only odd/even indices. We did all experiments three times and report the average results with the 95 percent confidence intervals.

7.1 Impact of N and I

We first evaluate the performance of eDiscovery for different combinations of N and I , using Constant as the baseline. During a single experiment, we run the continuous Bluetooth inquiry on one phone and eDiscovery on another simultaneously. We conducted the experiments in and around the Stamp Student Union of the University of Maryland. We walked along a pre-defined route for around 30 minutes during the experiments. Most of the Bluetooth devices discovered by us should be on mobile phones, although they can also be on other mobile devices such as tablets and laptops.

We plot the percentage of discovered Bluetooth devices of eDiscovery and Constant in Fig. 6. We also summarize their estimated energy consumption in Table 5. The experimental results show that increasing N and I can save

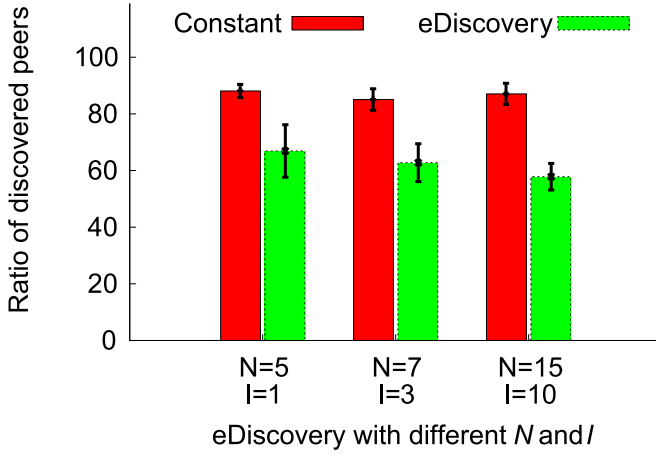


Fig. 6. The ratio of the number of discovered peers for Constant and eDiscovery to the ground truth, with different N and I . The error bars are the 95 percent confidence intervals.

smartphone energy consumption at the expense of a higher missing probability. When $N = 5$ and $I = 1$, eDiscovery consumes only 56 percent energy of Constant, and discovers 21 percent less peers than it. These results also partially verify experimentally the theoretical analysis by Wang et al. [29] that the probing scheme with constant inquiry intervals achieves the minimum discovery-missing probability among all probing methods with the same average inquiry interval. The ratio of discovered peers between Constant and the ground truth is higher than 80 percent for all experiments.

7.2 Dynamic Environment

We then compare the performance of eDiscovery ($N = 5$ and $I = 1$) with Constant and STAR [29] in three different environments: the Student Union of the University of Maryland, the Union Station of Washington D.C. and the Mall at Short Hills in New Jersey. We also chose a pre-defined route in the other two locations, including both indoor and outdoor environments, and the duration of experiments was about 30 minutes too. Generally, there are much more peers in the indoor environment than the outdoor environment in these three locations. We limit the inquiry interval of STAR to be 10-200 seconds, the same as eDiscovery.

We plot in Fig. 7 the percentage of discovered peers of eDiscovery, Constant and STAR, compared with the ground truth. In each group of experiments, we run Constant along with either eDiscovery or STAR. Thus there are two bars for Constant for each location in Fig. 7. We note that the first two bars in this figure for Constant and eDiscovery present the results from the same experiment

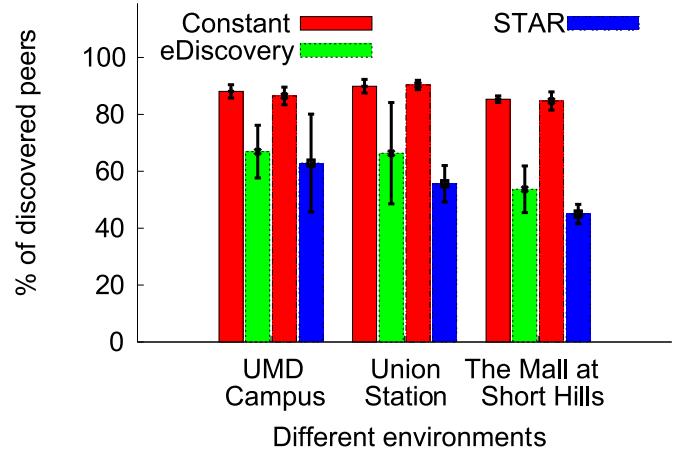


Fig. 7. The comparison of the percentage of discovered peers for different schemes. The error bars are the 95 percent confidence intervals.

as the first two bars in Fig. 6. We also plot in Fig. 8 the energy consumption of eDiscovery and STAR, compared with Constant. As we can see from these figures, eDiscovery performs better than STAR in all three locations. In particular, eDiscovery discovers more peers than STAR but consumes much less energy on smartphones.

eDiscovery outperforms STAR for two reasons. First, eDiscovery takes into account not only the inquiry interval, but also the duration of inquiry, to further reduce smartphone energy consumption. As shown in Section 4, the active probing state consumes much more energy than the idle state of Bluetooth inquiry. Second, it adapts to environmental changes (i.e., the number of neighboring peers) much more quickly than STAR, which is important in dynamic environments.

7.3 An in-Depth Look at the Traces

To verify the above, we took an in-depth look at the traces collected for the experiments we did in the Mall at Short Hills. We plot the start time, duration, and the number of discovered peers of a single experiment for STAR in Fig. 9a and for eDiscovery in Fig. 9b. For a Bluetooth device discovery starting at s and ending at t that discovers p peers, we plot a horizontal bar from (s, p) to (t, p) . We note that in

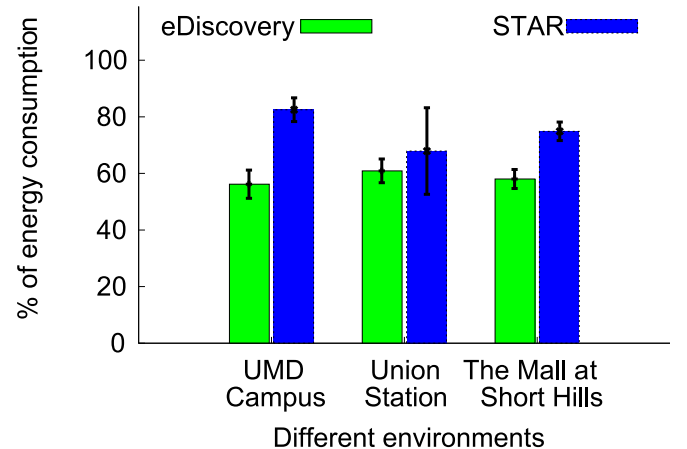


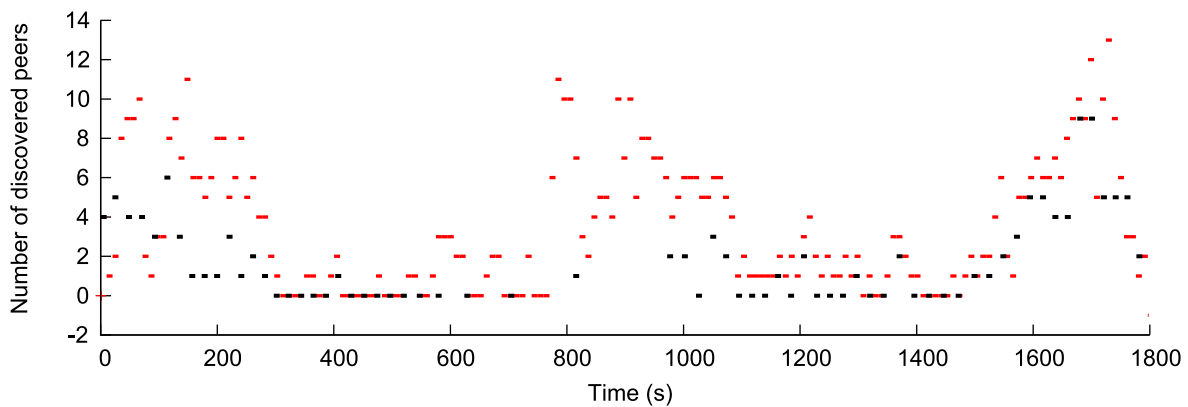
Fig. 8. The comparison of the percentage of energy consumption for different schemes, using Constant as baseline. The error bars are the 95 percent confidence intervals.

TABLE 5
The Estimated Energy Consumption (in Joules) of eDiscovery with Different N and I and the Comparison with Constant

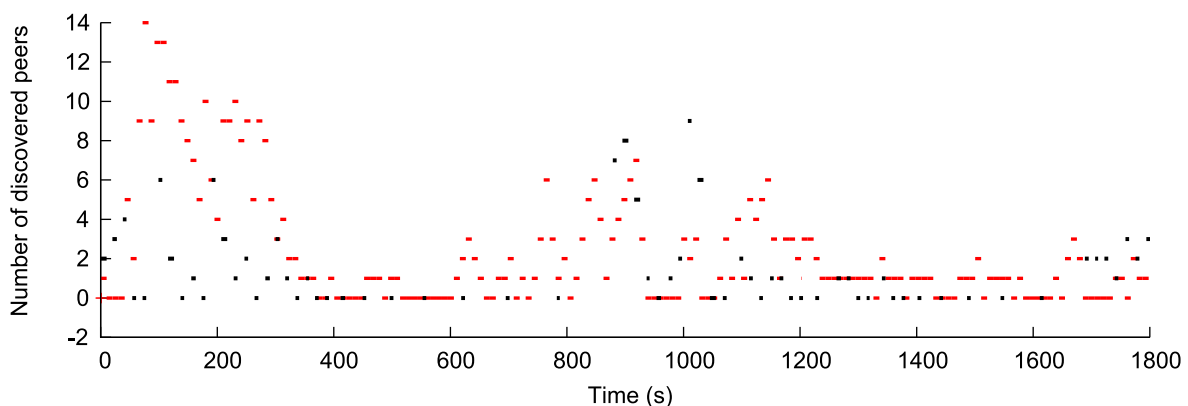
Param. N, I	Constant	eDiscovery	Percent
5, 1	220.83 (212.67, 228.99)	123.93 (115.92, 131.94)	56.12%
7, 3	209.02 (202.63, 215.41)	113.84 (96.24, 131.44)	54.46%
15, 10	210.80 (200.86, 220.74)	105.60 (103.70, 107.50)	50.09%

The numbers in the parentheses are the 95 percent confidence intervals.

Authorized licensed use limited to: Columbia University Libraries. Downloaded on July 06, 2026 at 02:36:33 UTC from IEEE Xplore. Restrictions apply.



(a) The start time, duration and the number of discovered peers of Bluetooth device discovery for a STAR experiment. The red horizontal bars are for the ground truth and the black ones are for STAR.



(b) The start time, duration and the number of discovered peers of Bluetooth device discovery for an eDiscovery experiment. The red horizontal bars are for the ground truth and the black ones are for eDiscovery.

Fig. 9. Detailed traces of eDiscovery and STAR experiments.

these two figures, a Bluetooth device may be counted several times if it appeared in multiple device discoveries. In each figure, we use the red color to plot the ground truth and the black color for either STAR or eDiscovery. During both experiments, we discovered more than 100 peers in the ground truth. The percentage of discovered peers is around 60 percent for eDiscovery and 40 percent for STAR.

There are two main observations from Figs. 9a and 9b. First, on average the duration of Bluetooth device discovery in eDiscovery is shorter than STAR (6.79 seconds versus 10.25 seconds), which is demonstrated by the narrower black bars in Fig. 9b. Second, eDiscovery increases the inquiry intervals much faster than STAR when there are few peers and decreases the intervals much quicker when there are more peers. For example, from 300 to 600 seconds of both experiments, there were at most three peers found by each inquiry. During this quiet period, eDiscovery performed Bluetooth inquiry only 10 times, three times less than STAR. Moreover, during the period from 800 to 1,000 seconds when there were more peers, eDiscovery performed Bluetooth inquiry seven times, four times more than STAR. On the one hand, the shorter discovery duration and less frequent Bluetooth device discovery during the quiet period translate into less energy consumption of eDiscovery than STAR. On the other hand, the more frequent device discovery when there are many peers is one of the

reasons that the discovery-missing probability of eDiscovery is lower than STAR.

7.4 Comparison with Another Protocol

We also evaluate the performance of the RAL protocol in Drula et al. [9], by measuring the number of discovered peers. RAL can discover only less than 30 percent of peers found in the ground truth for the experiment we did in the Mall at Short Hills. The possible reason may be that even for the most aggressive discovery mode in RAL, the duration of Bluetooth device discovery is less than 1 second, which is too short to complete a scan of all possible Bluetooth frequency bands. Differently from RAL, the shortest duration of Bluetooth device discovery in eDiscovery is 5.12 seconds, which is more suitable when the number of neighboring peers changes dynamically. Note that although it is possible to tune the parameters of RAL and STAR to improve their performance, that is beyond the scope of this paper.

7.5 Summary

To summarize, our performance evaluation shows that if energy consumption is not a major concern and the key objective is to discover more neighboring devices, Constant may be a good choice. It can discover more than 80 percent peers but consumes only half energy of

TABLE 6
Performance Evaluation of eDiscovery
Using Different Parameters

Param. / Value		% of Discovered Peers	Duration of Inquiry
Default		74.39 (73.96, 74.82)	581.33 (579.30, 583.36)
$base_W$	4	47.48 (46.92, 48.04)	323.71 (321.42, 326.00)
	12	74.72 (74.29, 75.15)	585.69 (583.64, 587.74)
$base_I$	6	76.62 (76.18, 77.06)	693.54 (690.79, 696.29)
	14	68.42 (67.99, 68.85)	490.58 (488.96, 492.20)
$small_W$	3	47.47 (46.91, 48.03)	328.24 (325.91, 330.57)
	7	79.40 (79.03, 79.77)	748.66 (746.76, 750.56)
inc_NP	5	80.47 (80.16, 80.78)	628.25 (627.05, 629.45)
	15	69.40 (68.88, 69.92)	541.01 (538.15, 543.87)
p	0.9	74.91 (74.50, 75.32)	585.56 (583.62, 587.50)
	0.7	74.57 (74.13, 75.01)	580.35 (578.25, 582.45)
α	0.5	74.51 (74.10, 74.92)	581.30 (579.31, 583.29)
	1.0	74.73 (74.32, 75.14)	578.49 (576.53, 580.45)
	1.5	73.22 (72.80, 73.64)	565.07 (562.94, 567.20)

The numbers in the parentheses are the 95 percent confidence intervals.

continuous device discovery. However, when the major goal is to save energy on smartphones and the missing of some peers is acceptable, we should use eDiscovery to dynamically tune the parameters of Bluetooth device discovery. In other words, the selection between Constant and eDiscovery depends on the requirements of applications that actually use them for device discovery.

8 SIMULATION STUDIES IN NS-2

To perform a much more extensive evaluation of eDiscovery, we port its implementation into the ns-2 simulator enhanced with the UCBT Bluetooth module (version 0.9.9.2a). This UCBT module is for Bluetooth version 1.2 and there is no significant difference between the device discovery specifications of Bluetooth versions 1.2 and 2.1.

8.1 eDiscovery Parameters

Using UCBT based simulation studies, we evaluate how the parameters of Algorithm 1, including different values of $base_W$, $base_I$, $small_W$ and inc_NP , and the choice of random variable r , affect the performance of eDiscovery. Recall that we set the default values of these parameters as listed in Table 4. The simulation setup is as follows. The simulation area is a $1,800 \times 20$ rectangle. The inquiring Bluetooth devices moves from (0, 10) to (1,800, 10) with a constant speed 1 m/s and thus the simulation duration is 30 minutes. We distribute 100 scanning Bluetooth devices in the simulation area uniformly and randomly.

We summarize the simulation results for 1,000 randomly generated network topologies in Table 6. The second row of this table shows the simulation results with the default values of these parameters. For the rest of the table, when the value of a parameter is changed, we use the default values for all other parameters. The major observation from Table 6 is that by increasing inquiry duration or decreasing inquiry interval eDiscovery can discover more peers, but at the cost of longer inquiry time. Based on the description about energy consumption of Bluetooth device discovery in Section 4.4, the

TABLE 7
Performance Evaluation of eDiscovery and STAR
in the ns-2 Simulator

	% of Discovered Peers	Duration of Inquiry
eDiscovery	77.80 (77.32, 78.28)	410.00 (408.21, 411.79)
STAR	75.83 (74.93, 76.73)	676.23 (671.40, 681.06)

The numbers in the parentheses are the 95 percent confidence intervals.

longer inquiry time will lead to higher energy consumption. Moreover, the performance of eDiscovery is more sensitive to the change of inquiry duration. Compared with the default setup, decreasing the value of $base_W$ by 4, or the value of $small_W$ by 2 will reduce the device discovery probability by 36 percent (whereas increasing their values by the same amount can discover about only 0.4 and 6.7 percent more peers). In eDiscovery, we use the two key parameters N and I to dynamically control the values of inquiry duration and interval.

Another observation from Table 6 is that the performance of eDiscovery does not heavily depend on the choice of r which is determined by the probability p , because the mean of r is always 0 no matter how large or small p is. We verified this with very small values of p . For example, when $p = 0.1$, eDiscovery can find 1.6 percent more peers and increase inquiry duration by 4.3 percent. Moreover, increasing α from 0.5 to 1.0 has limited impact on the performance of eDiscovery. However, when changing from 1.0 to 1.5, eDiscovery discovers about 2 percent less peers and reduces the inquiry duration by around 2 percent.

8.2 Comparison of eDiscovery and STAR

To offer a direct apple-to-apple comparison of eDiscovery and STAR and evaluate their performance for more network topologies, we also port the implementation of STAR into the UCBT Bluetooth module. We summarize the simulation results for 1,000 randomly generated network topologies in Table 7. The simulation setup is similar to that in Section 8.1. To validate the experimental results in Section 7, we distribute Bluetooth devices in the simulation area based on the characteristics of our collected traces. More specifically, we divide the area into five regions and the device density of regions 1, 3 and 5 is much higher than that of regions 2 and 4, similar to the distribution illustrated in Fig. 9. We set the Bluetooth communication range to be 10 meters.

The two metrics that we are interested in are the percentage of discovered peers and the duration of Bluetooth inquiry. As we can see from Table 7, although eDiscovery discovers only slightly more peers than STAR, the standard deviation of the percentage of discovered peers is much smaller for eDiscovery than STAR. Moreover, the duration of Bluetooth inquiry in eDiscovery is only around 60 percent of that of STAR, which confirms the energy-efficiency feature of eDiscovery because shorter inquiry time means less energy consumption. Both eDiscovery and STAR discover more peers in the simulations than in the field studies. One of the possible reasons may be that there is no co-channel interference considered in the ns-2 simulator. When two Bluetooth devices are in the communication range of each other (one of them is in the

TABLE 8
Performance Evaluation of eDiscovery and STAR
in the ns-2 Simulator With Interlaced Inquiry Scan

	% of Discovered Peers	Duration of Inquiry
eDiscovery	85.55 (85.12, 85.98)	417.70 (416.09, 419.31)
STAR	82.50 (81.58, 83.42)	689.33 (684.91, 693.75)

The numbers in the parentheses are the 95 percent confidence intervals.

inquiring mode and another in the scanning mode), the discovery probability is very close to 1.0, which is not true in practice. This high device discovery probability in the ns-2 simulator also decreases eDiscovery's room for improvements.

In addition to the standard inquiry scan mode described in Section 2, Bluetooth version 2.1 also introduces an optional interlaced inquiry scan mode to increase the discovery probability. When in the interlaced inquiry scan mode, a Bluetooth device performs two back to back scans, where the first one is on the normal hop frequency f_{scan} and the second one is on frequency $(f_{scan} + 16) \bmod 32$. This means that the two inquiry scan frequencies will be in different trains.

It is hard to evaluate the performance of device discovery protocols with the interlaced inquiry scan mode in practice because by default Bluetooth devices use the standard inquiry scan mode and it is impossible to change this setting on the discovered mobile phones in our field studies. Thus, we also evaluate the performance of eDiscovery and STAR in the ns-2 simulator with the interlaced inquiry scan mode enabled and report the simulation results in Table 8. By comparing Table 8 with Table 7, we can see that interlaced inquiry scans can increase the number of discovered peers, but at the same time also increase the duration of Bluetooth inquiry. Still, eDiscovery outperforms STAR when the interlaced inquiry mode is enabled.

9 DISCUSSION

In this section, we discuss the limitations of this paper and some possible extensions of eDiscovery.

9.1 Bluetooth Low Energy

As pointed out by Liu et al. [19], although the wide-range parameter settings of Bluetooth LE (in Section 2.2) offer the flexibility for devices to customize the discovery performance, improper settings could significantly increase the latency and energy consumption of Bluetooth LE device discovery. The device discovery procedure for Bluetooth LE looks simpler than that of the classical Bluetooth (e.g., smaller number of channels and the elimination of switching between two trains). However, they share fundamentally the same design principle of interleaving between the transmission of multiple inquiry messages/advertising packets and staying in the idle mode to save energy. We can extend our proposed scheme about how to dynamically change the duration and interval of classical Bluetooth inquiry to control the duration of advertising events (i.e., the number of advertising packets to send out) and the *advInterval* in a similar way, and thus further reduce the energy consumption

for device discovery in Bluetooth LE. We leave this extension as our future work.

9.2 Other Extensions

Device discovery is only the first step of opportunistic communications. The next two steps are service discovery and data transfer. There are several options of service discovery. We can exploit the standard service discovery protocol of Bluetooth [3], or develop our own protocols.

We plan to leverage multiple radio interfaces on smartphones, such as Bluetooth and WiFi, for opportunistic data transfer. These interfaces usually have different communication ranges and diverse radio characteristics. Pering et al. [22] have demonstrated the benefits of energy reduction by switching between these interfaces for mobile applications. In our case, Bluetooth may be suitable for short data transfer due to its low-power nature. For transmissions of large amounts of data, WiFi may be more desirable, because its data rate is higher and its communication range is much longer than Bluetooth. Although WiFi is not energy efficient for device discovery, we can still enable it for data transfer after mobile phones discover each other through Bluetooth.

9.3 Limitations

Although we have evaluated the performance of eDiscovery in three different realistic environments, the major limitation of the evaluation is that we had no control of other mobile phones during the experiments. If all the mobile phones perform Bluetooth device discovery, the number of phones discovered by us may be changed, as Bluetooth devices that are in inquiry state at the same time cannot discover each other [14], [23]. During our field experiments, most of the discovered Bluetooth devices were probably in discoverable mode only. Running experiments on mobile testbeds, such as CrowdLab [8], may solve this problem.

Another limitation of our work presented in this paper is that we have evaluated the performance of eDiscovery on only Nokia N900 smartphones. We are planing to port eDiscovery to other smartphone platforms, such as Android and iPhones, and evaluate its performance on them.

10 CONCLUSION

In this paper, we present eDiscovery, an adaptive device discovery protocol for reducing energy consumption of smartphone-based opportunistic communications. To choose the underlying communication technology, we measured the power of Bluetooth and WiFi device discovery on Nokia N900 and HTC Hero smartphones. Based on the measurement results, we prefer Bluetooth to WiFi because Bluetooth is more energy efficient for device discovery. eDiscovery dynamically changes the Bluetooth inquiry duration and interval to adapt to dynamic environments. We verify the effectiveness of eDiscovery through the first experimental field study of Bluetooth device discovery in three different environments, using a prototype implementation on smartphones. We are currently working on a more extensive evaluation of eDiscovery to further improve its performance.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers for their insightful comments. They thank Min Mao and Yu Xiao for their help on measurement setup. They also thank Pan Hui for his support with devices and equipment. The work of B. Han and A. Srinivasan was supported in part by the US National Science Foundation (NSF) under ITR Award CNS-0426683, NSF Award CNS-0626636, and NSF Award CNS-1010789. The work of J. Li was supported in part by the National Basic Research Program of China under Grants 2011CBA00300 and 2011CBA00301 and the National Natural Science Foundation of China under Grants 61202009, 61033001, 61061130540, and 61073174. This work was done when B. Han was a graduate student at the University of Maryland.

REFERENCES

- [1] L. Aalto, N. Göthlin, J. Korhonen, and T. Ojala, "Bluetooth and WAP push based location-aware mobile advertising system," in *Proc. 2nd Int. Conf. Mobile Syst., Appl. Services*, Jun. 2004, pp. 49–58.
- [2] M. Bakht, M. Trower, and R. H. Kravets, "Searchlight: Won't you be my neighbor?" in *Proc. 18th Annu. Int. Conf. Mobile Comput. Netw.*, Aug. 2012, pp. 185–196.
- [3] Bluetooth Special Interest Group, "Specification of the bluetooth system, Version 2.1 + EDR," Jul. 2007.
- [4] Bluetooth Special Interest Group, "Specification of the bluetooth system, version 4.0," Jun. 2010.
- [5] G. Chakraborty, K. Naik, D. Chakraborty, N. Shiratori, and D. Wei, "Analysis of the bluetooth device discovery protocol," *Wireless Netw.*, vol. 16, no. 2, pp. 421–436, Feb. 2010.
- [6] R. Cohen and B. Kapchits, "Continuous neighbor discovery in asynchronous sensor networks," *IEEE/ACM Trans. Netw.*, vol. 19, no. 1, pp. 69–79, Feb. 2011.
- [7] M. Conti, S. Giordano, M. May, and A. Passarella, "From opportunistic networks to opportunistic computing," *IEEE Commun. Mag.*, vol. 48, no. 9, pp. 126–139, Sep. 2010.
- [8] E. Cuervo, P. Gilbert, B. Wu, and L. P. Cox, "CrowdLab: An architecture for volunteer mobile testbeds," in *Proc. 3rd Int. Conf. Commun. Syst. Netw.*, Jan. 2011, pp. 1–10.
- [9] C. Drula, C. Amza, F. Rousseau, and A. Duda, "Adaptive energy conserving algorithms for neighbor discovery in opportunistic bluetooth networks," *IEEE J. Sel. Areas Commun.*, vol. 25, no. 1, pp. 96–107, Jan. 2007.
- [10] P. Dutta and D. Culler, "Practical asynchronous neighbor discovery and rendezvous for mobile sensing applications," in *Proc. 6th ACM Conf. Embedded Netw. Sensor Syst.*, Nov. 2008, pp. 71–83.
- [11] R. Friedman, A. Kogan, and Y. Krivolapov, "On power and throughput tradeoffs of wifi and bluetooth in smartphones," in *Proc. INFOCOM*, Apr. 2011, pp. 900–908.
- [12] S. Gollakota, F. Adib, D. Katabi, and S. Seshan, "Clearing the RF Smog: Making 802.11 robust to cross-technology interference," in *Proc. ACM SIGCOMM*, Aug. 2011, pp. 170–181.
- [13] M. Grossglauser and D. N. C. Tse, "Mobility increases the capacity of Ad Hoc wireless networks," *IEEE/ACM Trans. Netw.*, vol. 10, no. 4, pp. 477–486, Aug. 2002.
- [14] B. Han, P. Hui, V. S. A. Kumar, M. V. Marathe, J. Shao, and A. Srinivasan, "Mobile data offloading through opportunistic communications and social participation," *IEEE Trans. Mobile Comput.*, vol. 11, no. 5, pp. 821–834, May 2012.
- [15] B. Han and A. Srinivasan, "eDiscovery: Energy efficient device discovery for mobile opportunistic communications," in *Proc. IEEE Int. Conf. Netw. Protocols*, Oct./Nov. 2012, pp. 1–10.
- [16] D. B. Johnson and D. A. Maltz, "Dynamic source routing in ad hoc wireless networks," in *Mobile Computing*. Norwell, MA, USA: Kluwer, 1996, pp. 153–181.
- [17] A. Kandhalu, K. Lakshmanan, and R. R. Rajkumar, "U-Connect: A low-latency energy-efficient asynchronous neighbor discovery protocol," in *Proc. 9th ACM/IEEE Int. Conf. Inform. Process. Sensor Netw.*, Apr. 2010, pp. 350–361.
- [18] M. Liberatore, B. N. Levine, and C. Barakat, "Maximizing transfer opportunities in bluetooth DTNs," in *Proc. ACM CoNEXT Conf.*, Dec. 2006, pp. 1–11.
- [19] J. Liu, C. Chen, and Y. Ma, "Modeling and performance analysis of device discovery in bluetooth low energy networks," in *Proc. IEEE Global Commun. Conf.*, Dec. 2012, pp. 1538–1543.
- [20] M. J. McGlynn and S. A. Borbash, "Birthday protocols for low energy deployment and flexible neighbor discovery in Ad Hoc wireless networks," in *Proc. 2nd ACM Int. Symp. Mobile Ad Hoc Netw. Comput.*, Oct. 2001, pp. 137–145.
- [21] L. McNamara, C. Mascolo, and L. Capra, "Media sharing based on colocation prediction in urban transport," in *Proc. ACM 14th ACM Int. Conf. Mobile Comput. Netw.*, Sep. 2008, pp. 58–69.
- [22] T. Pering, Y. Agarwal, R. Gupta, and R. Want, "CoolSpots: Reducing the power consumption of wireless mobile devices with multiple radio interfaces," in *Proc. 4th Int. Conf. Mobile Syst., Appl. Services*, Jun. 2006, pp. 220–232.
- [23] B. S. Peterson, R. O. Baldwin, and J. P. Kharoufeh, "Bluetooth inquiry time characterization and selection," *IEEE Trans. Mobile Comput.*, vol. 5, no. 9, pp. 1173–1187, Sep. 2006.
- [24] N. Ristanovic, G. Theodorakopoulos, and J.-Y. L. Boudec, "Traps and pitfalls of using contact traces in performance studies of opportunistic networks," in *Proc. IEEE INFOCOM*, Mar. 2012, pp. 1377–1385.
- [25] T. Salonidis, P. Bhagwat, and L. Tassiulas, "Proximity awareness and fast connection establishment in bluetooth," in *Proc. 1st Annu. Workshop Mobile Ad Hoc Netw. Comput.*, Aug. 2000, pp. 141–142.
- [26] A. Sharma, V. Navda, R. Ramjee, V. N. Padmanabhan, and E. M. Belding, "Cool-Tether: Energy efficient on-the-fly wifi hot-spots using mobile phones," in *Proc. 5th Int. Conf. Emerging Netw. Experiments Technol.*, Dec. 2009, pp. 109–120.
- [27] T. J. Smith, S. Saroiu, and A. Wolman, "BlueMonarch: A system for evaluating bluetooth applications in the wild," in *Proc. 7th Int. Conf. Mobile Syst., Appl. Services*, Jun. 2009, pp. 41–53.
- [28] S. Vasudevan, D. Towsley, D. Goeckel, and R. Khalili, "Neighbor discovery in wireless networks and the coupon collector's problem," in *Proc. 15th Annu. Int. Conf. Mobile Comput. Netw.*, Sep. 2009, pp. 181–192.
- [29] W. Wang, V. Srinivasan, and M. Motani, "Adaptive contact probing mechanisms for delay tolerant applications," in *Proc. 13th Annu. Int. Conf. Mobile Comput. Netw.*, Sep. 2007, pp. 230–241.
- [30] X. Wu, S. Tavildar, S. Shakkottai, T. Richardson, J. Li, R. Laroia, and A. Jovicic, "FlashLinQ: A synchronous distributed scheduler for peer-to-peer ad hoc networks," in *Proc. 48th Annu. Allerton Conf. Commun. Control Comput.*, Sep./Oct. 2010, pp. 514–521.
- [31] Z. Yang, B. Zhang, J. Dai, A. Champion, D. Xuan, and D. Li, "E-SmallTalker: A distributed mobile system for social networking in physical proximity," in *Proc. IEEE 30th Int. Conf. Distrib. Comput. Syst.*, Jun. 2010, pp. 468–477.
- [32] W. Zhao, M. Ammar, and E. Zegura, "A message ferrying approach for data delivery in sparse mobile ad hoc networks," in *Proc. 5th ACM Int. Symp. Mobile Ad Hoc Netw. Comput.*, May 2004, pp. 187–198.

Bo Han received the bachelor's degree in computer science and technology from Tsinghua University in 2000, the MPhil degree in computer science from City University of Hong Kong in 2006, and the PhD degree in computer science from the University of Maryland in 2012. He is currently a senior member of technical staff at AT&T Labs Research. He interned at AT&T Labs Research in the summers of 2007, 2008, and 2009, Deutsche Telekom Laboratories for the summer of 2010, and HP Labs during the summer of 2011. His research interests are in the areas of wireless networking, mobile computing, software-defined networking and network functions virtualization, with a focus on developing simple yet efficient and elegant solutions for real-world networking and systems problems.

Jian Li received the BSc degree from Sun Yat-sen (Zhongshan) University, China, the MSc degree in computer science from Fudan University, China, and the PhD degree from the University of Maryland, US. He is an assistant professor at the Institute for Interdisciplinary Information Sciences, Tsinghua University. His research interests lie in the areas of algorithms, databases and wireless sensor networks. He has coauthored several research papers that have been published in major computer science conferences and journals. He received best paper awards at VLDB 2009 and ESA 2010.

Aravind Srinivasan received the PhD degree from Cornell University and the BTech degree from the Indian Institute of Technology, Madras. He is a professor at the Department of Computer Science and the Institute for Advanced Computer Studies at the University of Maryland, College Park. His research interests are in randomized algorithms, networking, social networks, and combinatorial optimization, as well as in the growing confluence of algorithms, networks, and randomness in society, in fields including the social web and public health. He has published several papers in these areas, in journals including *Nature*, *Journal of the ACM*, *IEEE/ACM Transactions on Networking*, and *SIAM Journal on Computing*. He is an editor of three journals, and has served on the program committees of various conferences. He is a fellow of the IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**